CINETFLIX

Autor: Dilan Alejandro González Alatriste
Matrícula: 202521115
Profesor: Jaime Alejandro Romero Sierra
Fecha de Entrega: 05 de marzo de 2026

Introducción

El presente documento constituye el **Reporte de Bitácora de Pruebas Técnicas** para el sistema **'CINETFLIX'**, un sistema de gestión y reservaciones cinematográficas desarrollado íntegramente en Python 3.x.

El diseño del proyecto se fundamenta en el paradigma de **Programación Orientada a Objetos (POO)**, con un énfasis particular en la robustez estructural a través de la **Herencia Múltiple**. Esta arquitectura fue seleccionada para garantizar la escalabilidad y la correcta separación de responsabilidades.

Un concepto clave de ingeniería implementado rigurosamente es la cadena de herencia: **`Persona`**, **`Empleado`**, **`Administrador`**. Esta estructura asegura que la clase **`Administrador`** herede las propiedades básicas de una persona y las capacidades de un empleado, añadiendo permisos y funcionalidades especiales para la gestión crítica del sistema.

Las pruebas a continuación se centran en la validación de la lógica de negocio central, incluyendo la gestión de transacciones (**pagos**, **puntos de lealtad**), la validación de **promociones** y el formateo de documentos finales como el **ticket** de compra.

Cronología de Desarrollo

A continuación, se detallan los hitos clave alcanzados durante el desarrollo de la arquitectura del sistema CINETFLIX.

Fecha	Hito de Desarrollo	Descripción del Avance
28 de Febrero	Definición de Estructura de Usuarios	Establecimiento de las clases base (e.g., **`Persona`**) y la herencia directa para entidades de usuario (e.g., **`Usuario`**).
01 de Marzo	Lógica de Empleados	Implementación de la clase **`Empleado`** y sus métodos de gestión de funciones internas.
03 de Marzo	Infraestructura de Proyección	Creación de clases de ubicación y contenido (e.g., **`Espacio`**, **`Sala`**, **`Película`**).
04 de Marzo	Refactorización de Roles	Implementación de la clase **`Administrador`**, que hereda de **`Empleado`**, para segregar permisos críticos del sistema.

Se inició con un código muy “simple”. Donde sólo se definian las clases y los métodos

```python
class Rol(Enum):
    TAQUILLERO = "TAQUILLERO"
    ADMINISTRACION = "ADMINISTRACION"
    LIMPIEZA = "LIMPIEZA"

class TipoSala(Enum):
    DOS_D = "2D"
    TRES_D = "3D"
    IMAX = "IMAX"

# -CLASES
# CLASES DE USUARIOS / NORMAL / ADMIN
class Persona:
    def __init__(self, id_persona: int, nombre: str, email: str, telefono: str):
        self.id_persona = id_persona
        self.nombre = nombre
        self.email = email
        self.telefono = telefono

    def login(self):
        """Simula la autenticación en el sistema"""
        print(f"Sesión iniciada: {self.nombre}")

    def logout(self):
        """Cierra la sesión del usuario o empleado"""
        print(f"Sesión cerrada: {self.nombre}")
```

Posteriormente se profundizó el código y mejoró gracias a la observación detallada del DIAGRAMA UML

```python
#SÁBADO 28 DE FEBRERO //////

#Definimos las clases persona y usuario que contiene la herencia de persona:

class Persona:
    def __init__(self, name, id, mail, telefono):
        self.name = name
        self.id = id
        self.mail = mail
        self.telefono = telefono

class Usuario(Persona):
    def __init__(self, name, id, mail, telefono, dinero, puntos):
        super().__init__(name, id, mail, telefono)
        self.dinero = dinero
        self.puntos = puntos
        self.historial_reservas = []

#Definimos los MÉTODOS

    def login(self):
        print(f"\n[LOGIN] Hola {self.name}, bienvenido a CINETLFIX!.")
        print(f"Tienes un crédito disponible de: ${self.dinero}")

    def logout(self):
        print(f"[LOGOUT] Nos vemos pronto {self.name}, gracias por visitarnos.")

    def deposito(self, monto):
        self.dinero += monto
        print(f"Depósito exitoso. Nuevo saldo: ${self.dinero}")

    def crear_reserva(self, reserva):
        self.historial_reservas.append(reserva)

```

```python
#DOMINGO 01 DE MARZO ####

# AHORA CLASES DE EMPLEASO (ADMIN) QUE HEREDA ATRIBUTOS DE LA CLASE PERSONA

class Empleado(Persona):
    def __init__(self, name, id, mail, telefono, trabajo):
        super().__init__(name, id, mail, telefono)
        self.trabajo = trabajo

## 04 DE MARZO CORRECCIÓN: me había faltado la clase ADMINISTRADOR DEL DIAGRAMA:
class Administrador(Empleado):
    def __init__(self, name, id, mail, telefono, trabajo, permisos):
        super().__init__(name, id, mail, telefono, trabajo)
        self.tipodepermisos = permisos




    def marcar_entrada(self):
        print(f"[NOMINA] {self.name} ha marcado su entrada.")

    def gestionar_funciones(self):
        if self.trabajo == "admin":
            print(f"[ADMIN] {self.name} está gestionando las funciones.")
        else:
            print("[!] Acceso denegado: Se requiere rol de admin.")

```

Se verificará la correcta instanciación de objetos, la funcionalidad de los métodos individuales clave y la integridad del flujo completo de reservación, asegurando que los principios de herencia y polimorfismo se cumplan conforme al diseño original.

Sección 1: Evidencia de Instanciación

Esta sección documenta la validación de la estabilidad del sistema mediante la instanciación masiva de objetos. El objetivo fue verificar la correcta asignación de memoria, la inicialización de atributos y la unicidad de identificadores.

Se procedió a instanciar 10 objetos para cada una de las entidades principales (**`Usuario`**, **`Empleado`**, **`Administrador`**, **`Producto`**, **`Sala`** y **`Película`**).

La prueba se considera exitosa al confirmarse en la consola que:

- Todos los objetos se crearon sin errores de memoria.
- Los atributos críticos (**`nombre`**, **`rol`**, **`tipo_sala`**) fueron inicializados correctamente mediante los constructores.
- La lógica de la clase **`Administrador`** (que hereda de **`Empleado`**) mantuvo la integridad de los atributos heredados al instanciar.

```
# --- INSTANCIACIÓN DE LOS 10 OBJETOS POR CATEGORÍA ---

# 10 Usuarios (Inspirados en tu banda y otros nombres nuevos)
u1 = Usuario("Dilan González", 1, "dilan@cinetflix.com", "222001", 1000, 500)
u2 = Usuario("Eduardo González", 2, "edu@mail.com", "222002", 450, 120)
u3 = Usuario("Gerry Bajista", 3, "gerry@rock.com", "222003", 600, 300)
u4 = Usuario("Richy Guitar", 4, "richy@rock.com", "222004", 150, 45)
u5 = Usuario("Pato Drummer", 5, "pato@rock.com", "222005", 800, 200)
u6 = Usuario("Elizabeth Amador", 6, "elyzama@mail.com", "222006", 300, 80)
u7 = Usuario("Oscar Téllez", 7, "oscar@mail.com", "222007", 500, 150)
u8 = Usuario("Sofía Galindo", 8, "sofix@mail.com", "222008", 200, 30)
u9 = Usuario("Andrés G.", 9, "andres@mail.com", "222009", 750, 400)
u10 = Usuario("Alejandra Rivera", 10, "ale@mail.com", "222010", 400, 90)
usuarios = [u1, u2, u3, u4, u5, u6, u7, u8, u9, u10]

# 10 Empleados (Mezcla de Administradores y Staff)
e1 = Administrador("Andrés González", 11, "director@cinetflix.com", "555101", "admin", "Control Total")
e2 = Empleado("Juan Pérez", 12, "juan@cinetflix.com", "555102", "taquillero")
e3 = Empleado("Marco Polo", 13, "marco@cinetflix.com", "555103", "limpieza")
e4 = Empleado("Luis Lopez Tello", 14, "Ranscellot@cinetflix.com", "555104", "taquillero")
e5 = Administrador("Dilan G. Admin", 15, "dilan_admin@cinetflix.com", "555105", "admin", "Gestión Pro")
e6 = Empleado("Fabiola Solís", 16, "fabi@cinetflix.com", "555106", "limpieza")
e7 = Empleado("Roberto Carlos", 17, "rober@cinetflix.com", "555107", "taquillero")
e8 = Administrador("Ignacio S. Redfield.", 18, "NACHCRICKO@cinetflix.com", "555108", "admin", "Soporte Técnico")
e9 = Empleado("Saúl Gorgonio.", 19, "saul@cinetflix.com", "555109", "limpieza")
e10 = Empleado("Marlene V.", 20, "marlenene@cinetflix.com", "555110", "taquillero")
empleados = [e1, e2, e3, e4, e5, e6, e7, e8, e9, e10]
```

```python
# 10 Salas Cinetflix
s1 = Sala("Sala Premium 1", "IMAX", 150, True)
s2 = Sala("Sala 2", "3D", 80, False)
s3 = Sala("Sala 3", "2D", 120, False)
s4 = Sala("Sala 4 DX", "4DX", 60, True)
s5 = Sala("Sala 5", "3D", 50, False)
s6 = Sala("Sala 6 Kidz", "2D", 100, False)
s7 = Sala("Sala 7", "2D", 90, False)
s8 = Sala("Sala VIP 8", "IMAX", 40, True)
s9 = Sala("Sala 9", "3D", 55, False)
s10 = Sala("Sala 10 Lounge", "2D", 30, True)
salas = [s1, s2, s3, s4, s5, s6, s7, s8, s9, s10]

# 10 Zonas de Comida
c1 = Comida("Lobby Central", 100, ["Palomitas", "Refresco"])
c2 = Comida("Zona Express", 50, ["Hot Dog", "Nachos"])
c3 = Comida("Coffee Corner", 30, ["Café", "Donas"])
c4 = Comida("Cinetflix Bar", 20, ["Cocteles", "Botanas"])
c5 = Comida("Dulcería 2", 80, ["Gomitas", "Chocolates"])
c6 = Comida("Pizza Point", 40, ["Pizza", "Refresco"])
c7 = Comida("Ice Cream Shop", 25, ["Helado", "Malteada"])
c8 = Comida("VIP Lounge Snacks", 15, ["Canapés", "Vino"])
c9 = Comida("Kiosko Norte", 60, ["Papas", "Agua"])
c10 = Comida("Snack Estelar", 70, ["Palomitas", "Jugos"])
zonas = [c1, c2, c3, c4, c5, c6, c7, c8, c9, c10]

# 10 Películas Cinetflix
p1 = Pelicula("Data Science: The Movie", 130, "Documental", "A")
p2 = Pelicula("Mala Idea: El Concierto", 110, "Musical", "A")
p3 = Pelicula("Invasión Python", 145, "Sci-Fi", "B")
p4 = Pelicula("El Código Perdido", 120, "Misterio", "B")
p5 = Pelicula("Algoritmo Mortal", 105, "Terror", "C")
p6 = Pelicula("La Red Social 2", 115, "Drama", "B")
p7 = Pelicula("Cyberpunk 2077: PueblaYork", 160, "Acción", "C")
p8 = Pelicula("El Viaje de la IA", 95, "Animación", "A")
p9 = Pelicula("Base de Datos Sangrienta", 140, "Thriller", "C")
p10 = Pelicula("Pandas al Rescate", 85, "Comedia", "A")
peliculas = [p1, p2, p3, p4, p5, p6, p7, p8, p9, p10]
```

Sección 2: Pruebas de Métodos Individuales

Se ejecutaron pruebas unitarias sobre métodos específicos para validar la lógica interna del negocio y la interacción con librerías externas.

Clase	Método Probado	Funcionalidad Clave	Resultado de la Prueba
Pago	confirmar_pago()	Valida el **`saldo`** del usuario y gestiona la acumulación de **`puntos_lealtad`**.	El método devolvió **`True`** para transacciones con saldo suficiente y el objeto **`Usuario`** correspondiente incrementó sus puntos de lealtad en 10%.
Promoción	es_valida()	Uso de la librería **`datetime`** para verificar si la promoción se encuentra vigente.	La prueba con una fecha actual (FECHA) y una fecha de caducidad pasada resultó en **`False`**, confirmando la lógica de vigencia.
Reserva	generar_ticket()	Formatea y presenta los datos de la compra para el cliente final (incluye nombre del cine y fecha actual).	Se verificó la correcta salida de datos con el formato impreso.

Anexo de capturas:

```python
    def confirmar_pago(self):
        if self.usuario.dinero >= self.total:
            self.usuario.dinero -= self.total
            self.usuario.puntos += int(self.total // 10)
            self.estado = "Confirmada"
            self.usuario.crear_reserva(self)
            print(f"Pago exitoso. Total: ${self.total}")
        else:
            print("Saldo insuficiente.")
```

```python
class Reserva:
    contador = 1
    def __init__(self, usuario, funcion, asientos):
        self.id = Reserva.contador
        Reserva.contador += 1
        self.usuario = usuario
        self.funcion = funcion
        self.asientos = []
        self.estado = "Pendiente"

        for a in asientos:
            if self.funcion.sala.asiento_libre(a):
                self.asientos.append(a)
                self.funcion.sala.asientos_ocupados.append(a)

        self.total = len(self.asientos) * self.funcion.precio
```

```python
class Promocion:
    def __init__(self, codigo, descripcion, descuento, fecha_fin):
        self.codigo = codigo
        self.descripcion = descripcion
        self.descuento = descuento
        self.fecha_fin = fecha_fin

    def es_valida(self):
        # Comparamos la fecha de vencimiento con el día de hoy
        if self.fecha_fin >= date.today():
            print(f"La promoción {self.codigo} es válida.")
            return True # Retornamos True para usarlo en la lógica de pago
        else:
            print(f"La promoción {self.codigo} ha vencido.")
            return False # Retornamos False para usarlo en la lógica de pago

class Comida:
    def __init__(self, lugar, capacidad, productos):
        self.lugar = lugar
        self.capacidad = capacidad
        self.productos = productos

    def disponibilidad(self):
        print(f"Zona: {self.lugar} | Capacidad: {self.capacidad} personas.")
```

[Captura de Pantalla: Método generar_ticket()]

Como prueba de la correcta implementación del método **`generar_ticket()`** de la clase **`Reserva`**, se adjunta una representación de la salida formateada para el cliente final. Este método imprime el nombre del cine y utiliza **`date.today()`** para la fecha actual, garantizando la trazabilidad del documento.

```
    def generar_ticket(self):
        if self.estado == "Confirmada":
            print(f"\n--- TICKET #{self.id} ---")
            print(f"Cine CINETFLIX")
            print(f"Fecha: {date.today()}")
            print(f"Usuario: {self.usuario.name}")
            print(f"Película: {self.funcion.pelicula.titulo}")
            print(f"Horario: {self.funcion.horario}")
            print(f"Asientos: {', '.join(self.asientos)}")
            print(f"Total: ${self.total}")
        else:
            print("La reserva no ha sido confirmada.", {self.funcion.pelicula.titulo})


    def mostrar_detalle(self):
        return f"Producto: {self.nombre}, Precio: ${self.precio}, Categoría: {self.categoria}"
```

Conclusiones

Las pruebas técnicas confirman la solidez del sistema *CINETFLIX* y validan la correcta implementación de los principios de Programación Orientada a Objetos, particularmente la **Herencia Múltiple** en la jerarquía **`Persona`** , **`Empleado`** , **`Administrador`**.

Retos Superados:

1. **Gestión de Asientos:** Se implementó y validó con éxito una lógica para el control de asientos ocupados, previniendo la doble venta de ubicaciones.
2. **Errores de Indentación:** Se superaron los desafíos iniciales de sintaxis de Python relacionados con la indentación en estructuras de control y la definición de clases anidadas.

El sistema está listo para avanzar a la fase de pruebas de estrés y rendimiento.

Referencias

El código completo del proyecto CINETFLIX y la documentación técnica detallada se encuentran disponibles en el repositorio oficial del proyecto.

Link al Repositorio Oficial: https://github.com/DAGA-Mx/SISTEMA-DE-CINE-CINETFLIX-